Part III: Dynamic Programming

Lecture 10: The 0-1 Knapsack Problem

Computer Science and Technology
United International College

Outline

Introduction to Part III I

What is dynamic programming?

- A technique for solving optimization problems.

What are optimization problems?

- Problems with many possible solutions.
- Each solution has a value.
- Objective: Find a solution with the optimal value.

Introduction to Part III II

How to develop a dynamic programming? Four steps

1. Structure: Analyze structure of an optimal solution, and thereby choose a space of subproblems.
2. Recursion: Establish relationship between the optimal value the problem and those of some subproblems
3. Bottom-up computation:
   - Compute the optimal values of the smallest subproblems first, save them in the table,
   - Then compute optimal values of larger subproblems, and so on, until the optimal value of the original problem is computed.
4. Construction of optimal solution: Assemble optimal solution by tracing the computation at the previous step.

Notes:

- Steps 1 and 2 are related.
- Step 4 is not always necessary: we sometimes need only the optimal value.

Introduction to Part III III

Dynamic programming (DP) vs Divide-and-Conquer (DC)

- Commonalities:
  1. Partition the problem into particular subproblems.
  2. Solve the subproblems.
  3. Combine the solutions to solve the original one.
- Differences:
  - DC:
    - Efficient when the subproblems are independent.
    - Not efficient when subproblems share subsubproblems.
    - Some subproblems might be solved many times.
  - DP:
    - Suitable when subproblems share subsubproblems.
    - Some each subproblem only once.
    - The result is stored in a table in case it is needed elsewhere.
    - DP trades space for time.

Introduction to Part III IV

We will study three examples of Dynamic programming:

- The 0-1 Knapsack Problem
- Chain Matrix Multiplication
- All Pairs Shortest Path

Outline

0-1 Knapsack Problem (Informal Description)

We have n items.

- v_i denotes the value of the ith item
- w_i denotes the weight of the ith item
- You are given a knapsack capable of holding total weight W.

Goal: Use the knapsack to carry items, such that the total values are maximum

- we want to find a subset of items to carry such that
  - The total weight is at most W.
  - The total value of the items is as large as possible.

We cannot take parts of items, it is the whole item or nothing. (This is why it is called 0-1.)

Question

How should we select the items?

0-1 Knapsack Problem (Formal Description)

Definition

Given $W > 0$, and two n-tuples of positive numbers $< v_1, v_2, ..., v_n >$ and $< w_1, w_2, ..., w_n >$, we wish to determine the subset $T \subseteq \{1, 2, ..., n\}$ (of items to carry) that

$$\text{maximizes} \sum_{i \in T} v_i$$

$$\text{subject to} \sum_{i \in T} w_i \leq W$$

Remark: This is an optimization problem. The brute force solution is to try all 2^n possible subsets T.

Question

Is there a better way?

Ans: Yes. Dynamic Programming!

Outline

Developing a DP Algorithm for Knapsack I

Step 1: Space of Subproblems

- For $1 \leq i \leq n$, and $0 \leq w \leq W$, define

$$V[i, w] = max\left\{ \sum_{j \in T} v_j : T \subseteq \{1, 2, ..., i\}, \sum_{j \in T} w_j \leq w \right\}$$

  The maximum (combined) value of any subset of items $\{1, 2, ..., i\}$ of (combined) weight at most w.

- Note that our problem is: find $V[n, W]$

Terms:

- T is a solution for $[i, w]$ if $T \subseteq \{1, 2, ..., i\}$ and $\Sigma_{j \in T} w_j \leq w$
- T is a optimal solution for $[i, w]$ if T is a solution and $\Sigma_{j \in T} v_j \leq V[i, w]$.

Developing a DP Algorithm for Knapsack II

Step 2: Value of a problem and those of its subproblems

Recursion: Intuitively, an optimal solution would either choose item i is or not choose item i.

$$V[i, w] = max\{V[i-1, w], v_i + V[i-1, w-w_i]\}$$

for $1 \leq i \leq n$, $0 \leq w \leq W$.

- Optimal structure: optimal solution for problem contains optimal solutions to subproblems. Indication that DP might apply.
- The recursion is why we chose the problem space $\{v[i, w] : 1 \leq i \leq n, 0 \leq w \leq W\}$ at the first place.

Boundary cases:

- Set

$$\begin{array}{lll} V[0, w] = 0 & \text{for } 0 \leq w \leq W, & \text{no item} \\ V[i, w] = -\infty & \text{for } w < 0, & \text{illegal} \end{array}$$

Developing a DP Algorithm for Knapsack III

Step 3: Bottom-up computation of $V[i, w]$

Recurrence:

$$V[i, w] = max\{V[i-1, w], v_i + V[i-1, w-w_i]\}$$

We compute and save $v[i, w](1 \leq i \leq n, 0 \leq w \leq W)$ in such an order that: When it is time to compute $V[i, w]$, the values of $V[i-1, w]$ and $V[i-1, -w_i]$ are available.

So, we fill the following table row by row and left to right.

V[i, w]	w = 0	1	2	3	...	...	W
i = 0	0	0	0	0	...	...	0
1	→	→	→	→	→	→	→
2	→	→	→	→	→	→	→
...	→	→	→	→	→	→	→
n	→	→	→	→	→	→	→

bottom ↓

Example of the Bottom-up computation

Example

Let $W = 10$ and

i	1	2	3	4
v_i	10	40	30	50
w_i	5	4	6	3

$V[i, w]$	0	1	2	3	4	5	6	7	8	9	10
$i = 0$	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	10	10	10	10	10	10
2	0	0	0	0	40	40	40	40	40	50	50
3	0	0	0	0	40	40	40	40	40	50	70
4	0	0	0	50	50	50	50	90	90	90	90

Computation of some of the entries:

$$V(1,0) = max\{V(0,0), v_1 + V(0,-5)\} = max\{0, -\infty\} = 0$$
$$V(1,5) = max\{V(0,5), v_1 + V(0,0)\} = max\{0, 10+0\} = 10$$
$$V(2,4) = max\{V(1,4), v_2 + V(1,0)\} = max\{0, 40+0\} = 40$$
$$V(2,9) = max\{V(1,9), v_2 + V(1,5)\} = max\{10, 40+10\} = 50$$

The final output is: $V[4, 10] = 90$

The Dynamic Programming Algorithm I

Algorithm 1 KnapSack(v, w, n, W)

```
1:  for w=0 to W do
2:      V[0,w]=0
3:  end for
4:  for i=1 to n do
5:      for w=0 to W do
6:          if w[i] ≤ w then
7:              V[i, w] = max{V[i − 1, w], v_i + V[i − 1, w − w[i]]}
8:          else
9:              V[i,w] = V[i-1,w]
10:         end if
11:     end for
12: end for
13: return V[n,W]
```

Time complexity: Clearly, $\mathcal{O}(nW)$.

The Dynamic Programming Algorithm II

Rewriting the algorithm slightly:

Algorithm 2 KnapSack(v, w, n, W)

```
1:  for w=0 to W do
2:      V[0,w]=0
3:  end for
4:  for i=1 to n do
5:      for w=0 to W do
6:          if (w[i] ≤ w) and (v[i]+V[i−1, w−w[i]] > V[i−1, w]) then
7:              V[i, w] = v_i + V[i − 1, w − w[i]]
8:          else
9:              V[i,w] = V[i-1,w]
10:         end if
11:     end for
12: end for
13: return V[n,W]
```

Outline

Constructing the Optimal Solution I

- The algorithm 2 for computing $V[i, w]$ does not record which subset of items gives the optimal solution.
- To compute the actual subset, we can add an auxiliary boolean array $keep[i, w]$ which is
  - 1 if we choose item i in $V[i, w]$ and
  - 0 otherwise.

Question

How do we use all the values $keep[i, w]$ to determine the subset T of items having the maximum value?

Constructing the Optimal Solution II

If $keep[n, W]$ is 1, then $n \in T$. We can now repeat this argument for $keep[n-1, W-w_n]$.
If $keep[n, W]$ is 0, then $n \notin T$. and we repeat the argument for $keep[n-1, W]$.
Therefore, the following partial program will output the elements of T:

```
1: K = W
2: for i=n downto 1 do
3:     if Keep[i, K] == 1 then
4:         output i
5:         K = K-w[i]
6:     end if
7: end for
```

Example of optimal solution construction

Example

Let $W = 10$ and

i	1	2	3	4
v_i	10	40	30	50
w_i	5	4	6	3

$V[i, w]$	0	1	2	3	4	5	6	7	8	9	10
$i = 0$	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0(0)	0	10	10	10	10	10	10
2	0	0	0	0	40	40	40	40(1)	40	50	50
3	0	0	0	0	40	40	40	40(0)	40	50	70
4	0	0	0	50	50	50	50	90	90	90	90(1)

$keep[i, k]$ parentheses:

$$V(4, 10) = max\{V(3, 10), v_4 + V(3, 7)\} = max\{70, 50 + 40\} = 90, keep(4, 10) = 1$$
$$V(3, 7) = max\{V(2, 7), v_3 + V(2, 1)\} = max\{40, 30 + 0\} = 40, keep(3, 7) = 0$$
$$V(2, 7) = max\{V(1, 7), v_2 + V(1, 3)\} = max\{10, 40 + 0\} = 40, keep(2, 7) = 1$$
$$V(1, 3) = max\{V(0, 3), v_1 + V(0, -2)\} = max\{0, 0 - \infty\} = 0, keep(1, 3) = 0$$

Optimal solution: $\{4, 2\}$

The Complete Algorithm for the Knapsack Problem

Algorithm 3 KnapSack(v, w, n, W)

```
1: for w=0 to W do
2:     V[0,w]=0
3: end for
4: for i=1 to n do
5:     for w=0 to W do
6:         if (w[i] ≤ w) and (v[i] + V[i − 1, w − w[i]] > V[i − 1, w]) then
7:             V[i,w] = v[i]+V[i-1,w-w[i]]
8:             Keep[i,w] = 1
9:         else
10:            V[i,w] = V[i-1,w]
11:            Keep[i,w] = 0
12:        end if
13:    end for
14: end for
15: K = W
16: for i=n downto 1 do
17:    if Keep[i, K] == 1 then
18:        output i
19:        K = K-w[i]
20:    end if
21: end for
22: return V[n,W]
```

Outline

Dynamic Programming vs. Simple Recursion I

A simple recurison algorithm for 0-1 knapsack:

$$V[i, w] = max\{V[i-1, w], v_i + V[i-1, w-w_i]\}$$

Algorithm 4 KnapSackSR(v, w, n, W)

```
1: if n=0 then
2:     return 0
3: end if
4: if W ≤ 0 then
5:     return −∞
6: end if
7: V1 =KnapsackSR(v, w, n-1, W)
8: V2 =KnapsackSR(v, w, n-1, W-w[n])
9: return max{V1, v[n] + V2}
```

Dynamic Programming vs. Simple Recursion II

- Suppose $w[i] = 1$ for all i.
- Function calls and repetition of computation:
  - Entry Level: $V(n, W)$
  - Level 1: $V(n-1, W)$, $V(n-1, W-1)$
  - Level 2: $V(n-2, W)$, $V(n-2, W-1)$; $V(n-2, W-1)$, $V(n-2, W-2)$
  - ...
- Running time:
  - Recurrence:

$$f(n, W) \geq f(n-1, W) + f(n-1, W-1)$$

  - If $W \geq n$, we have

$$f(n, W) = \Omega(2^n)$$

Dynamic Programming vs. Divide-and-Conquer

- Dynamic programming saves us from having to recompute previously calculated subsolutions!
- The DP algorithm runs in $\mathcal{O}(nW)$ time.
- The simple recursion algorithm runs in $\Omega(2^n)$ time.